

We will address each of the above scalability requirements using the best practices in that area.

Best practices in Web app scalability

Practice# 1: Ensure that an application can handle traffic surges by scaling compute resources.

The solution is to scale compute resources. There are two types of scalability – vertical and horizontal.

Vertical scalability is when the application is running in one server and the server's resources are increased, e.g., increased RAM, a better CPU or a much bigger HDD. All such resources can lead to the better performance of an application, but vertical scalability has the following drawbacks:

- Downtime is required in order to scale up the machine.
- With all eggs in one basket – if something happens to this ultimate machine, the business is at risk!
- How much can a single machine scale anyway? There are limits to scale the resources in a machine.

Horizontal scaling on the other hand is bliss — one can put in identical resources (in this case, identical machines with the same application server) together and let them handle the traffic. Whenever traffic grows, just add up more identical resources together and let them handle the traffic via load balancers.

To segregate the load to application server clusters, the load balancer will be required in the front of the cluster, which will balance the load based on certain algorithms like Round-Robin, or IP Affinity, to direct the traffic towards the machines behind the load balancers. There are many open source software based load balancers. A few popular ones that are easy to work with are Apache (a module for load balancing is available as `mod_proxy_balancer`), Nginx (also has a balancer built-in), Pound, Squid (which is also a proxy server), or Inlab.de's Balance.

Advantages:

1. Scale up as required by adding more compute resources and reduce those resources to scale down.

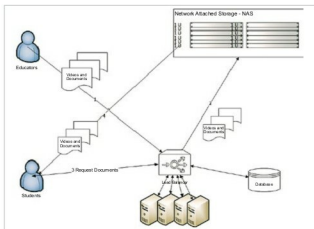


Figure 3: Web application with a load balanced application server

2. Cost saving: Scaling on a need basis and using optimum compute resources ensures that spending too will be optimum on compute resources.

Disadvantages:

1. Deploying and switching builds becomes a concern in a scaled out environment. Hence, a good DevOps practice is the need of the hour (one may have to use continuous deployment).

Practice #2: Ensure heavy duty processing while uploading content by the content producer and a seamless experience for content consumers, using 'shared nothing architecture'.

To achieve this, we need to consider a multi-tier deployment of the application workload. Consider that every application component is deployed onto its own dedicated server, which helps each component to handle the user load separately. This will increase the robustness of the application and provide a better experience to the end user (content producer/content consumer).

Advantages:

- Selected components can scale independently as the load increases on the respective component.
- Division of the workload will lead to division of the user load, ensuring the application can handle the user load efficiently.
- Less resources will be required to deploy the specific component, resulting in a lower cost of primary servers and of additional servers (in case of scale-out).

Disadvantages:

- Managing multiple tiers/servers will be a challenge. This requires one to have better control over the deployed blueprint.

The architecture of the application in this approach is depicted in Figure 4.

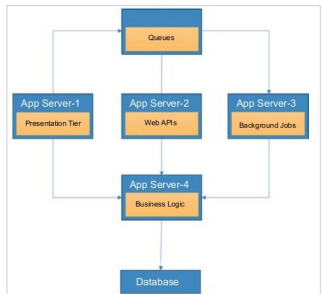


Figure 4: Application deployment in multi-tier architecture